

kneser: a precomputed evaluator for Kneser's real-analytic half-exponential

Guanghao Li¹

¹ Independent researcher

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

kneser evaluates the real-analytic *half-exponential* — the function f with

$$f(f(x)) = e^x$$

that Kneser proved to exist, to be real-analytic, and to be strictly increasing on all of $\mathbb{R}$ (Kneser, 1950) — together with the objects it is built from: the superexponential (tetration) sexp , its inverse the superlogarithm slog , and continuous iteration $\exp^{[t]}$ for arbitrary real t . These are not elementary functions and have no closed form; f is a genuinely transcendental object whose only practical definition is numerical.

The package is deliberately an *evaluator*, not a solver. A 50-significant-digit Taylor series of sexp at the origin is precomputed and shipped inside the package, so a call costs microseconds rather than seconds:

```
>>> import kneser
>>> kneser.half_exp(0.5)
1.0016400378866632
>>> kneser.half_exp(kneser.half_exp(0.5)) # = e^0.5
1.648721270700128
>>> kneser.exp_iter(2.0, 0.25) # exp iterated a quarter time
2.579802043276193
>>> kneser.hp.half_exp("0.5", dps=50) # 50-digit path
mpf('1.0016400378866631889882297295807994303276834552788')
```

Two numerical paths are exposed and never mixed implicitly: a float/stdlib path and an mpmath (Johansson & the mpmath development team, 2023) path under kneser.hp. The shipped coefficients are not a magic constant — `python -m kneser.build --digits N` regenerates them from scratch at any target precision, and the test suite passes against a freshly built table.

Statement of need

The mathematics of Kneser's construction is settled. Kneser gave the original Riemann-mapping argument (Kneser, 1950); Kouznetsov produced the first systematic high-precision computation (Kouznetsov, 2009); Trappmann and Kouznetsov gave a uniqueness criterion satisfied by Kneser's solution (Trappmann & Kouznetsov, 2010); Paulsen and Cowgill settled the resulting uniqueness conjecture and reached errors below 10^{-50} using 180 nodes (Paulsen & Cowgill, 2017), with extensions to the half-iterate of e^x specifically (Paulsen, 2016) and to complex bases (Paulsen, 2019).

What is missing is not another construction but ordinary availability. A researcher in complexity theory who needs a concrete half-exponential bound, or anyone wanting to plot exp[t] for a few dozen values of t, currently has three options: read digits out of a published table; install PARI/GP and run fatou.gp (Levenstein, 2010); or re-run a contour-integration solve at import time. None of these is a pip install away, and none returns a value in microseconds. kneser fills that gap. The expensive part of the problem — a theta-mapping iteration that takes several minutes to converge to 50 digits — is done once, offline, and its result is a data file in the package. What users import is a lookup-and-Horner evaluator with an mpmath escape hatch, a documented precision envelope, and a reproducible path back to the construction.

Comparison with existing software

Table with 4 columns: Metric, fatou.gp (Levenstein, 2010), kneser-iteration (Cooley, 2025), kneser (this work). Rows include Language Coefficients, Precision, Half-iterate, and Cost per call.

kneser-iteration (Cooley, 2025) is the closest neighbour and solves a genuinely different problem: it is a general KneserIterator engine for fractional iteration of analytic maps with suitable complex fixed points, computing its contour integrals afresh. kneser targets base e only, and trades that generality for a precomputed table, arbitrary precision, and constant-time evaluation. The two are complementary, and their values agree to float64 precision at the points where both are defined.

Implementation

The builder follows the theta-mapping approach (Kouznetsov, 2009; Levenstein, 2010): linearise exp at its complex fixed point L ≈ 0.318 + 1.337i to obtain a superfunction accurate to O(|L|^-DEPTH); correct it with a 1-periodic theta(z), retaining only the Fourier modes that decay in the upper half-plane — this selection is what picks out Kneser's real solution from the infinitely many C^infinity ones; then resample the Taylor series of sexp through a Cauchy integral on the unit circle, and iterate, gaining roughly 1.4 digits per pass. Every internal parameter (series depth, working precision, number of Taylor terms and Fourier modes) is derived from the requested digit count.

Verification

The test suite (52 tests) checks the defining functional equations on both numerical paths, the group law exp[s] o exp[t] = exp[s+t], exact recovery of exp and log at integer orders, and analytic identities that the construction is not told about — notably the chain rule f'(f(0)) · f'(0) = 1 and the closed form f'(0) = sexp'(-1/2)/sexp'(-1) = 0.87633613... Residuals are reported over a stated interval rather than claimed globally:

```
$ python -m kneser --verify --digits 50
max |f(f(x)) - e^x| on [-2, 2] = 1.81e-52 (dps=50)
```

Independent agreement to better than 10^{-47} is checked against a separate implementation written from the same algorithm family in a sibling research repository. Known limitations — float64 overflow of sexp near $z \gtrsim 4.4$, ill-conditioning approaching the logarithmic singularity at $z = -2$, and loss of *relative* accuracy for the composed identity on the far negative tail, where $f(x)$ collapses onto its finite asymptote $-0.696024740886 \dots$ — are documented with tested ranges in docs/precision-envelope.md rather than left for users to discover.

Acknowledgements

The theta-mapping iteration implemented here follows the algorithm developed by Sheldon Levenstein (sheldonison) and refined in the Tetration Forum community (Levenstein, 2010).

References

- Cooley, A. P. (2025). *kneser-iteration: Knese's method for fractional tetration in Python*. GitHub repository. <https://github.com/apcooley/kneser-iteration>
- Johansson, F., & the mpmath development team. (2023). *mpmath: a Python library for arbitrary-precision floating-point arithmetic*. <https://mpmath.org/>
- Kneser, H. (1950). Reelle analytische Lösungen der Gleichung $\varphi(\varphi(x)) = e^x$ und verwandter Funktionalgleichungen. *Journal Für Die Reine Und Angewandte Mathematik*, 187, 56–67. <https://doi.org/10.1515/crll.1950.187.56>
- Kouznetsov, D. (2009). Solution of $F(z+1) = \exp(F(z))$ in complex z -plane. *Mathematics of Computation*, 78(267), 1647–1670. <https://doi.org/10.1090/S0025-5718-09-02188-7>
- Levenstein, S. (2010). *fatou.gp: a fast, accurate Knese sexp algorithm for PARI/GP*. Tetration Forum, thread 486. <https://tetrationforum.org/showthread.php?tid=486>
- Paulsen, W. (2016). Finding the natural solution to $f(f(x)) = \exp(x)$. *Korean Journal of Mathematics*, 24(1), 81–106. <https://doi.org/10.11568/kjm.2016.24.1.81>
- Paulsen, W. (2019). Tetration for complex bases. *Advances in Computational Mathematics*, 45(1), 243–267. <https://doi.org/10.1007/s10444-018-9615-7>
- Paulsen, W., & Cowgill, S. (2017). Solving $F(z+1) = b^{F(z)}$ in the complex plane. *Advances in Computational Mathematics*, 43(6), 1261–1282. <https://doi.org/10.1007/s10444-017-9524-1>
- Trappmann, H., & Kouznetsov, D. (2010). Uniqueness of holomorphic Abel functions at a complex fixed point pair. *Aequationes Mathematicae*, 81(1), 65–76. <https://doi.org/10.1007/s00010-010-0021-6>